

1. Dynamic Array from Scratch

- a. En el git :3
- b. Invariante 1: $0 \leq \text{size} \leq \text{capacity}$ y $\text{capacity} \geq 1$

El arreglo físico nunca es más pequeño que lo que `size` promete, y nunca tiene tamaño cero. Se protege con el `if (size == capacity) resize(...)` antes de crecer, y con `if (size == 0) throw` antes de decrecer.

Invariante 2: `data.length == capacity`

El arreglo Java real siempre mide exactamente lo que dice `capacity`. Se mantiene porque ambos se crean/actualizan juntos: en el constructor (`new Object[capacity]`) y en

`resize(data = newData; capacity = newCapacity);`.

Invariante 3: orden lógico preservado — `data[i]` es el elemento en posición `i`, para $i < \text{size}$

Nada reordena las celdas. `append` solo escribe en `data[size]` (al final), `resize` copia en el mismo orden $i \rightarrow i$, y `set` solo reemplaza una celda sin mover las demás.

Invariante 4: ninguna operación pública toca `data[i]` con $i \geq \text{size}$ (o $i < 0$)

Es una zona inhabilitada del arreglo (celdas reservadas pero no usadas) queda inaccesible. Se garantiza porque `get` y `set` pasan obligatoriamente por `checkIndex`, la única puerta hacia `data`.

c.

get(index): Verificar que la posición sea válida es solo comparar números, y acceder a esa posición en un arreglo es directo, sin pasar por las demás casillas. No depende del tamaño de la estructura. Así que es $O(1)$.

set(index,value): Mismo principio que `get`: se verifica la posición y se sobrescribe directamente esa casilla, sin mover ni recorrer nada más. Así que es $O(1)$.

append(value): Si hay espacio libre, es $O(1)$: solo se coloca el valor al final. Pero si el arreglo está lleno, hay que reservar uno más grande y copiar **todos** los elementos existentes, uno por uno (no se permite copia optimizada del lenguaje). El número de elementos a copiar crece igual que el tamaño de la estructura, así que ese escenario es lineal. Como sí puede ocurrir (cada vez que el arreglo se llena), el peor caso queda definido por él: **$O(n)$** .

removeLast(): Sólo libera la última posición ocupada y reduce el contador; no desplaza nada porque elimina justo del extremo final. Al estar diseñada para **no** reducir la capacidad, tampoco aparece aquí un escenario costoso como el de `append`. Así que es $O(1)$

- d. **Worst-case** mide el costo de **una llamada aislada**. Cuando el arreglo está lleno, `append` dispara `resize`, que copia los n elementos existentes uno por uno — eso realmente cuesta $O(n)$, y es innegable. **Amortizado** mide el costo **promedio por llamada**, a lo largo de una secuencia larga de operaciones. Aquí el resultado es $O(1)$. **Por qué no se contradicen:** el `resize` no ocurre en cada llamada, solo cuando el arreglo se llena. Como la capacidad se **duplica** cada vez, después de un `resize` que deja capacidad C , hacen falta C llamadas baratas ($O(1)$ cada una) antes de que vuelva a dispararse el siguiente `resize`. Es decir: siempre hay tantas llamadas baratas "antes" como elementos a copiar "en" el próximo `resize`.

Demostración breve (método agregado): en n llamadas de `append` desde vacío, el costo total es: Escrituras normales: n pasos. Copias de todos los `resizes`: $1 + 2 + 4 + \dots + n/2 < 2n$ (serie geométrica).

Total: $O(n)$ para las n llamadas $\rightarrow$ costo promedio por llamada $= O(n)/n = O(1)$. El worst-case $O(n)$ describe correctamente el peor momento de una llamada puntual, y ese costo nunca desaparece. Lo que agrega el análisis amortizado es que ese caso caro es tan poco frecuente —gracias a la duplicación de capacidad— que, repartido entre todas las llamadas de la secuencia, el promedio se mantiene constante. Ambas afirmaciones son ciertas al mismo tiempo porque responden preguntas distintas: una sobre una operación aislada, otra sobre el promedio de una secuencia.

2. Snake: una implementación aparentemente eficiente

- a. La complejidad temporal de ejecutar la función `REMOVE_TAIL()` es $O(k)$, donde k representa la longitud actual de la serpiente (`size`). Esto se debe a que el algoritmo ejecuta un bucle `for` con el índice i desde 1 hasta $\text{size} - 1$, realizando exactamente $k - 1$ iteraciones. En cada paso del bucle se ejecuta una reasignación mediante la instrucción `data[i - 1] <- data[i]`, la cual posee un costo constante de $O(1)$ dado que en un `DynamicArray` el acceso directo a índices y valores permite que la lectura y escritura mediante índices tomen tiempo constante. Así pues, el tiempo de ejecución satisface $T(k) = (k - 1) \cdot O(1) = O(k)$.
- b. La complejidad total de realizar m movimientos consecutivos manteniendo la longitud k constante es $O(m \cdot k)$. Para cada movimiento sin comer el sistema ejecuta una inserción `append()` con costo amortizado $O(1)$ para hacer un `addHead`, pero también es necesario hacer uso de `REMOVE_TAIL()` con costo $O(k)$, lo que resulta en un costo por movimiento de $O(1) + O(k) = O(k)$. Es decir, por cada movimiento tenemos $O(k)$. Suponiendo m movimientos concluimos $T(m, k) = \sum_{j=1}^m O(k) = m \cdot O(k) = O(m \cdot k)$.
- c. La afirmación es falsa. Aunque la función `append()` posee una complejidad de $O(1)$ amortizado, la función `MOVE()` ejecuta obligatoriamente la eliminación de la cola mediante `REMOVE_TAIL()`. Como se demostró previamente, realizar una secuencia de m movimientos requiere un tiempo total de $T(m, k) = O(m \cdot k)$. Aplicando método de agregado para deducir el análisis amortizado

faltante en $(CostoAmortizado = \frac{T(m,k)}{m} = \frac{O(m \cdot k)}{m} = O(k))$, observamos que el tiempo promedio por operación depende directamente de la longitud k , por lo que la función `MOVE()` no es $O(1)$ amortizada.

- d. El cuello de botella de esta representación se ubica en el bucle `for` del método `REMOVE_TAIL()` propuesto por el estudiante. `for` el cual ejecuta un desplazamiento masivo de elementos hacia la izquierda. La eliminación del elemento situado al inicio ($p_0 = TAIL$) fuerza al sistema a copiar y reescribir uno a uno los $k - 1$ elementos restantes para preservar la contigüidad en memoria, lo que degrada el rendimiento de cada movimiento a tiempo lineal $O(k)$ e incluso hace cada vez menos y menos eficiente la implementación a medida que k (el `size`) crece. Cosa que naturalmente sucede en un juego de serpiente.

3. Redesign Challenge

Se propone un **buffer circular** sobre un único arreglo nativo: `addHead` inserta al final de la ventana lógica y `removeTail` libera el inicio de la ventana, sin desplazar nunca los elementos intermedios. Esto satisface a la vez `addHead = O(1)` amortizado y `removeTail = O(1)` worst-case, y funciona aunque los elementos no ocupen posiciones físicas consecutivas de izquierda a derecha.

a) Variables de estado adicionales

Además de `data`, `size`, `capacity`, se necesita una única variable nueva:

front (`int`, con $0 \leq \text{front} < \text{capacity}$): posición física donde se encuentra el elemento de índice lógico 0 (la `TAIL` actual). Es el "ancla" que da sentido circular al arreglo.

No hace falta guardar la posición de la cabeza por separado: siempre es $(\text{front} + \text{size} - 1) \bmod \text{capacity}$. Así se respeta la restricción de usar "un número constante de variables auxiliares" (una sola variable nueva).

b) Mapeo índice lógico $\rightarrow$ posición física

Para todo $0 \leq i < \text{size}$:

phys(i) = (front + i) mod capacity

Esta función es una biyección entre $[0, \text{size})$ y una **ventana circular contigua** de `size` celdas que arranca en `front` y "da la vuelta" si se pasa de `capacity - 1`. Por diseño no se garantiza que `phys(0) < phys(size-1)` en el sentido lineal del arreglo.

Validez del índice físico. Como `front` y i son enteros no negativos y el operador `mod` en Java sobre operandos no negativos siempre devuelve un valor en $[0, \text{capacity})$, se tiene $0 \leq \text{phys}(i) < \text{capacity}$ para cualquier i válido, es decir, `phys(i)` siempre cae dentro de los límites físicos del arreglo.

c) Invariantes

1. $0 \leq \text{size} \leq \text{capacity}$, $\text{capacity} \geq 1$.
2. $0 \leq \text{front} < \text{capacity}$ en todo momento.
3. Para todo $0 \leq i < \text{size}$, el elemento lógico i está en `data[(front + i) mod capacity]`, y estas `size` posiciones son todas distintas entre sí (ventana circular contigua de longitud `size`).
4. `addHead` únicamente escribe en la celda física $(\text{front} + \text{size}) \bmod \text{capacity}$ (la siguiente libre de la ventana) y luego incrementa `size`; nunca reescribe ni desplaza una celda ya ocupada.
5. `removeTail` únicamente lee/limpia `data[front]` y avanza `front`; nunca toca otra celda.
6. Tras un `resize`, `front = 0` y las primeras `size` celdas físicas del nuevo arreglo coinciden exactamente, en el mismo orden, con las `size` celdas lógicas que existían antes del `resize`.

De (3)-(6) se sigue que `get`, `addHead` y `removeTail` son correctos (preservan siempre el orden lógico) y que ninguno de los dos últimos requiere recorrer las `size` celdas salvo por el `resize` ocasional dentro de `addHead`. Esto da `removeTail = O(1)` **worst-case** siempre, y `addHead = O(1)` **amortizado** (mismo argumento de duplicación de capacidad que en la Pregunta 1, aplicado sobre el tamaño de la ventana lógica y no sobre una posición fija del arreglo).

f) Traza

Estado inicial: `size = capacity = 2`, cola A, cabeza B $\rightarrow$ `data = [A, B]`, `front = 0`.

Convención: cada nueva cabeza se nombra con la siguiente letra (C, D, E, ...).

Mov.	Acción	Arreglo físico (_ = vacío)	front	size	capacity	¿Resize?
N1 (no come)	addHead(C): size==capacity → resize a 4 → [A,B,_,_], luego escribe C en (0+2)%4=2 → [A,B,C,_], size=3. removeTail(): quita data[0]=A, front=1 → [_ ,B,C,_]	[_ , B, C, _]	1	2	4	Sí
N2 (come)	addHead(D): (1+2)%4=3 → [_ ,B,C,D], size=3	[_ , B, C, D]	1	3	4	No
N3 (no come)	addHead(E): (1+3)%4=0 → [E,B,C,D], size=4. removeTail(): quita data[1]=B, front=2 → [E,_,C,D]	[E, _ , C, D]	2	3	4	No
N4 (no come)	addHead(F): (2+3)%4=1 → [E,F,C,D], size=4. removeTail(): quita data[2]=C, front=3 → [E,F,_,D]	[E, F, _ , D]	3	3	4	No
N5 (come)	addHead(G): (3+3)%4=2 → [E,F,G,D], size=4	[E, F, G, D]	3	4	4	No
N6 (no come)	addHead(H): size==capacity → resize a 8 , relineal. desde front=3: [D,E,F,G,_,_,_,_], front=0, luego escribe H en (0+4)%8=4 → [D,E,F,G,H,_,_,_], size=5. removeTail(): quita data[0]=D, front=1 → [_ ,E,F,G,H,_,_,_]	[_ , E, F, G, H, _ , _ , _]	1	4	8	Sí

4. Amortized Analysis

Sean m el número de movimientos y $g \leq m$ el número de manzanas consumidas. La capacidad inicial es 2 y se duplica cada vez que el arreglo está lleno. En cada movimiento sin comer, addHead ocurre antes que removeTail.

a) Método agregado

La longitud permanente empieza en 2 y aumenta una unidad por manzana, así que alcanza como máximo $2 + g$. En un movimiento sin comer, el tamaño aumenta transitoriamente en 1 antes de quitar la cola. Por ello:

$$n_{\max} \leq g + 3.$$

Los resizes copian $2, 4, 8, \dots, 2^r$ elementos, porque cada uno copia los elementos almacenados justo cuando size == capacity. Su costo total es:

$$2 + 4 + \dots + 2^r = 2^{r+1} - 2 < 2 \cdot 2^r.$$

El último resize ocurre con capacidad 2^r ; al insertar la nueva cabeza, el tamaño llega a $2^r + 1$. Entonces $2^r < n_{\max}$, y el total de copias es menor que $2n_{\max}$, es decir, $O(g + 1)$. Es una cota superior: con $g = 0$ puede ocurrir un resize, pero también puede no ocurrir ninguno.

b) Por qué el costo de los resizes es lineal

Cada término de la suma es la capacidad existente justo antes de un resize: el arreglo está lleno y se copia exactamente esa cantidad de elementos. Como las capacidades se duplican, la suma de copias es menor que el doble de su último término:

$$2 + 4 + \dots + 2^r < 2 \cdot 2^r < 2n_{\max}.$$

Así, las copias totales son lineales en el tamaño máximo almacenado.

c) Costo total y costo amortizado

Cada uno de los m movimientos hace un `addHead` de costo base 1 y, si no come, un `removeTail` de costo $O(1)$. El costo base es $O(m)$; los `resizes` cuestan $O(g + 1)$. Por tanto:

$$T(m, g) = O(m) + O(g + 1) = O(m),$$

pues $g \leq m$. Para $m \geq 1$, hay m llamadas a `addHead`, así que su costo amortizado por llamada es $T(m, g)/m = O(1)$. Una llamada que redimensiona puede costar más en el peor caso.

d) Papeles de m y g

m controla el trabajo base de los movimientos. g controla el crecimiento de la serpiente y, por ello, el tamaño máximo y las copias de los `resizes`. El costo se expresa como $O(m) + O(g + 1)$, que queda en $O(m)$ porque $g \leq m$.

e) Método del potencial

Definimos:

$$\Phi = \max(0, 2 \cdot \text{size} - \text{capacity}).$$

Siempre es no negativo y, al inicio ($\text{size} = \text{capacity} = 2$), vale 2. El costo amortizado es el costo real más el cambio de potencial.

- **addHead sin resize:** cuesta 1 y el potencial aumenta como máximo en 2; costo amortizado ≤ 3 .
- **addHead con resize:** antes, $\text{size} = \text{capacity} = C$, por lo que $\Phi = C$. Copiar C elementos y escribir la cabeza cuesta $C + 1$. Después, $\text{size} = C+1$, $\text{capacity} = 2C$ y $\Phi = 2$. Así, el costo amortizado es $(C + 1) + (2 - C) = 3$.
- **removeTail:** cuesta 1 y reduce o mantiene el potencial; costo amortizado ≤ 1 .

Las eliminaciones no dejan un `resize` sin fondos: nunca vuelven negativo el potencial. Si lo reducen, las inserciones posteriores lo reconstruyen. Cuando el arreglo vuelve a llenarse con capacidad C , el potencial es C , suficiente para cubrir la copia.

Sumando: hay m inserciones y como máximo m eliminaciones. Con potencial inicial 2 y final no negativo, el costo real total es como máximo:

$$3m + m + 2 = 4m + 2 = O(m).$$

Por tanto, `addHead` tiene costo amortizado $O(1)$ por llamada.

5. The Adversarial Snake

a) La cantidad de operaciones de `resize` y de elementos copiados depende exclusivamente de las veces que se alcanza la condición $\text{size} = \text{capacity}$. En ausencia de contracción, el tamaño de la serpiente solo se mantiene o aumenta, por lo que para alcanzar una longitud arbitraria pero fija n , dado un `capacity` y una política, ya se puede saber **cuántas** veces se tendrá la condición $\text{size} = \text{capacity}$; número independiente de los turnos necesarios para llegar a dichas condiciones. Por ende, el total de copias es constante e independiente del adversario. Para aumentar el número de copias, el adversario necesitaría tener poder sobre reducir `size` y `capacity` de la snake, con lo cual podría repetir varias veces bucles al llegar a $\text{size} = \text{capacity}$, producir una copia, reducir el `size`, reducir la `capacity`, aumentar el `size` y volver a satisfacer $\text{size} = \text{capacity}$ con los mismos valores.

Política	# Resizes	Total copias	Costo amortizado por crecimiento	Espacio sin usar tras <code>resize</code>
$C + 1$	$O(n)$	$O(n^2)$	$O(n)$	0
$2C$	$O(\log n)$	$O(n)$	$O(1)$	$C/2$
$\lceil 1.5C \rceil$	$O(\log n)$	$O(n)$	$O(1)$	$C/3$

f) La política A es inviable para valores grandes de n debido al costo amortizado $O(n)$ que conlleva. En cuanto a la política B y C, ambas ofrecen costo amortizado constante y difieren únicamente en la memoria desperdiciada inicialmente (33% vs 50%).